Introduction to Software Testing:

- Importance or need of software testing
- Differences between Manual and Automation Testing

Introduction to White Box Testing:

- Advantages and Disadvantages of White box testing
- Static Techniques: Informal Reviews, Walkthroughs, Technical Reviews, Inspection
- Dynamic Techniques or Structural Techniques
  - Statement Coverage Testing, Branch Coverage Testing, Path Coverage Testing, Conditional Coverage Testing, Loop Coverage Testing

****

What is Software Testing?

Software testing is a process of executing a program or application with the purpose of finding the software bugs. i.e. to check whether the actual results match with expected results.

Software testing is a process to evaluate the functionality of a software application with purpose to find whether the developed software met the specified requirements or not and identify the defects to ensure that the product is defect-free in order to produce a quality product. i.e. process of verifying and validating a software product.

Software testing also helps to identify errors, gaps or missing requirements in the actual requirements. It can be either done manually or using automated tools.

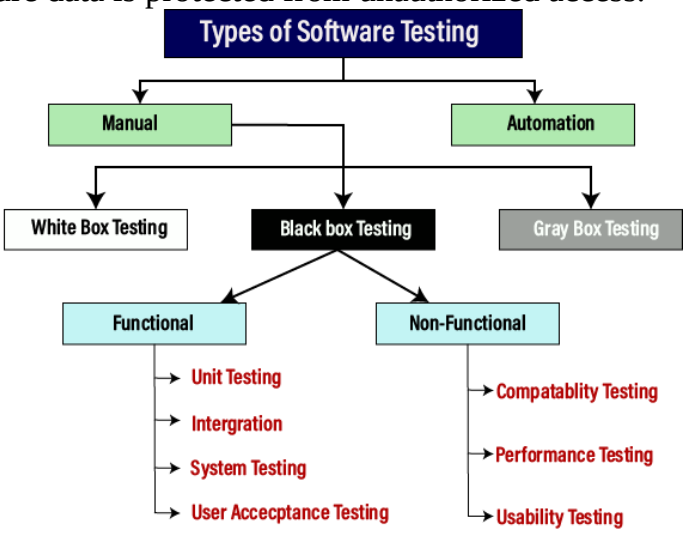
Example: Car manufacturer tests the car for maximum speed, fuel efficiency and safety from crash. These test later become the part of advertising policy for car sales.

Use or need of software testing

Software Testing is necessary because humans make mistakes all the time. Some of those mistakes are minor, some of expensive or dangerous. We need to check everything.

Software testing is very important because of the following reasons:

1. Find out the errors that are made during the development phases.
2. Early defect detection reduces cost of defect fixing.
3. To deliver high quality software
4. Low maintenance cost
5. Results should be more accurate, consistent and reliable.
6. To make sure that user requirements are fulfilled.
7. To ensure that no errors in the application code.
8. To ensure data is protected from unauthorized access.



➢ **Software Testing can be classified into two types:**

1. Manual Testing: Manual Testing is the process of testing the software manually. i.e. without using any automated tool. Testers use test plans, test cases, or test scenarios to test a software and give the final report about that software. Manual testing is a time consuming testing because it is done by humans and there is a chance of human errors.

2. Automation Testing: Usage of automation tools for executing test cases is known as Automation Testing. Automation Testing is used to re-execute test cases quickly, and repeatedly. Automation testing is also used to test the application from load, performance, and stress point of view. It increases the test coverage, improves accuracy, and saves time and money in comparison to manual testing. There is no chance of any human errors.

Who does Testing?

Software Testing can be done by all technical and non-technical people associated with the software. Testing in its various phases is done by-

- **Developer** –unit testing of the software and ensures that the individual methods work correctly.
- **Tester** – Testers are the face of software testing. A tester verifies the functionality of the application as a functional and performance test cases also automates the test cases.
- **Test Managers/Lead**– Makes test strategy and test plan documents.
- **End users** –User Acceptance Testing (UAT) of the application to make sure the software can work as per requirement or not.

Differences between Manual and Automation Testing

Manual Testing	Automation Testing
Test cases are executed manually.	Test cases are executed with the help of tools.
Reliability is less.	Reliability is more.
It is less costly.	It is more costly.
For some test cases it consumes time.	It takes less time to execute cases.
Human can make mistakes and accuracy is less	Machine rarely makes mistakes and high accuracy
Programming knowledge is not required to do manual testing.	Programming knowledge is a must in automation testing
It could be difficult to test the application on different Operating systems.	We can easily test the application on different Operating systems.
It becomes difficult to execute all the test cases and it impacts test coverage.	We can achieve the test coverage target.
Used for Exploratory, usability, ad-hoc testing	Used for Regression, performance, load testing.
Low quality and less reliable.	High quality and more reliable
In Manual Testing we find more bugs than automation by Error Guessing.	In Automation Testing we test the repetitive functionalities of the application.

Introduction to White Box Testing:

White box testing deals with internal logic or structure of code i.e. internal working of the code. In this testing, an internal structure, design, code of the application under test to verify input-output flow and to improve the design, security, and usability. White box testing is also known as Clear Box testing, Open Box testing, Structural testing, Transparent Box testing, Code-Based testing, and Glass Box testing.

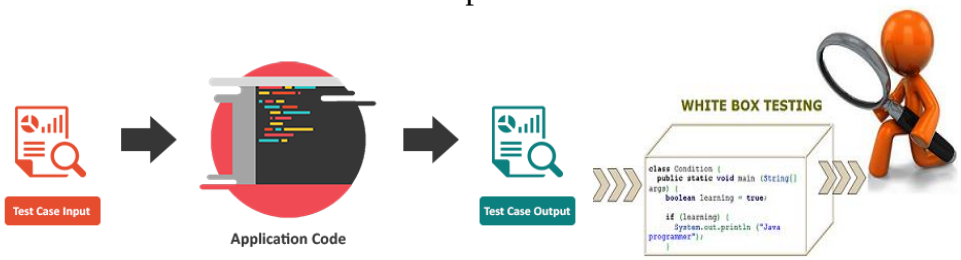
The Tester checks code and find out which unit/statement/portion of the code is faulty. The tester has test code line by line to ensure that internal operations are executed as per the specification and all internal modules are properly implemented. It does not focus on the functionality. The tests written based on the white box testing policy include coverage of the code written, branches, paths, statements and internal logic of the code etc. It mainly focuses on verifying the flow of inputs and outputs within the application. **WBT is mainly applied to Unit Testing.**

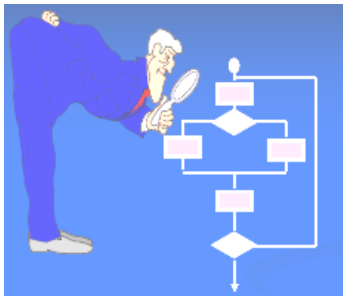
White box testing contains the testing of software code for:

- Checking for gaps in internal security
- Detecting incomplete or poorly structured paths and conditional loops in a code.
- The smooth flow of input data through the code.
- Expected output
- Proper functioning of conditional loops.
- Testing of each statement, object and function individually

It includes testing a series of predefined inputs against expected or desired outputs so that when a specific input does not result in the expected output, the developer will know that there is a bug.
The goal of white box testing is to:

- Guarantee that all independent paths within a module have been exercised at least once.
- Exercise all logical decisions on their true and false sides.
- Execute all loops at their boundaries and within their operational bounds.
- Exercise internal data structures to assure their validity.
- Exercise all data define and use paths.

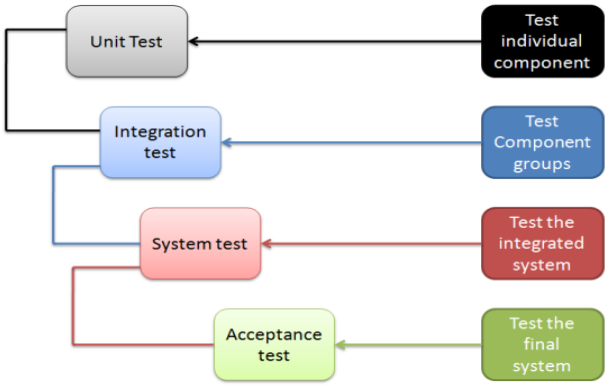


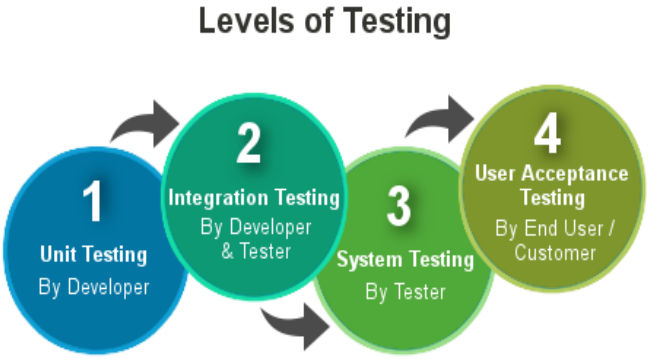


Example: White Box Testing is like the work of a mechanic who examines the engine to see why the car is not moving.

In White Box testing method is applicable to the following levels of software testing:

Four levels of software testing:	
Unit Testing	Individual units/components of a software/system are tested. The purpose is to validate that each unit of the software performs as designed.
Integration Testing	Individual units are combined and tested as a group. The purpose of this level of testing is to show faults in the interaction between integrated units.
System Testing	A testing process where a complete, integrated system is tested. The purpose of this testing is to evaluate the system completed with the specified requirements.
Acceptance Testing	To evaluate the system's completion with the business requirements and assess whether it is acceptable for delivery.





Advantages of white-box testing:

1. Code optimization by finding hidden errors in early phases of development
2. It can be started in the SDLC phase even without GUI.
3. Removes unnecessary lines of code
4. Thorough testing by covering all possible paths of a code
5. Test cases of white box testing can be easily automated.
6. Faster to find reason of failures
7. It is more accurate
8. Help tester to choose the best input data

➢ **Disadvantages of white-box testing**:

1. Requires code access
2. A highly skilled tester is required for testing who have deep programming knowledge

3. Time consuming and Costly
4. Changing code frequently.
5. It is not possible for testing every path/condition of software program its costly
6. White box testing can be quite complex.
7. Lack of system testing as user view point

➢ **How do you perform White Box Testing?**

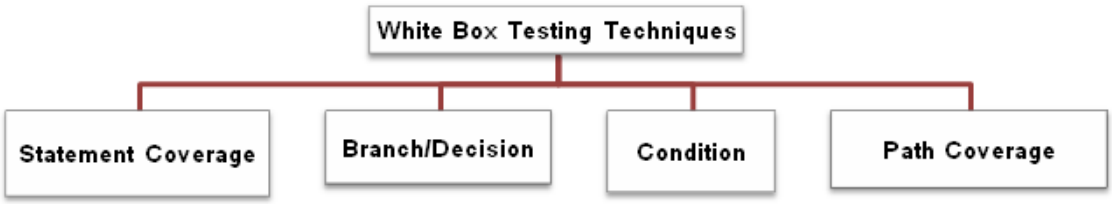
STEP 1) Understand the Source Code

- A tester learns and understands the source code of the application. The tester must be very knowledgeable in the programming languages.
- The tester should be able to find security issues and prevent attacks from hackers.

Step 2) Create Test Cases and Execute

- Writing more test cases for verify flow and structure of the application's source code.
- The tester will develop little tests for each process or series of processes in the application.

White box testing design techniques



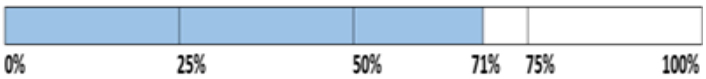
Statement coverage

- The statement coverage is also known as line coverage or segment coverage.
- The statement coverage **covers only the true conditions.**
- **Execute all the statements in the program at least once under some test.**
- The statement coverage can be calculated as:
- **Example:**

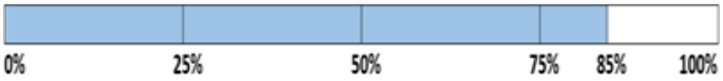
```
Read a;
Read b;
if a>b
Print "A is greater than B";
else
Print "B is greater than A";
end if;
```

$$\text{Statement coverage} = \frac{\text{Number of statements exercised}}{\text{Total number of statements}} \times 100\%$$

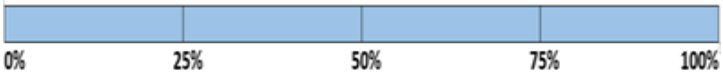
Test Case 1: If a=5 and b=1, then the first print statement is executed.
Number of executed statements = 5,
Total number of statements = 7
Statement Coverage: 5/7*100 = 71%



Test Case 2: If a=1 and b=5, then the second print statement is executed.
Number of statements executed=6,
Total number of statements in the code=7
Statement coverage - 6/7 * 100=85%



All the statements are covered by 2nd test case. So we can conclude that overall statement coverage is 100%.

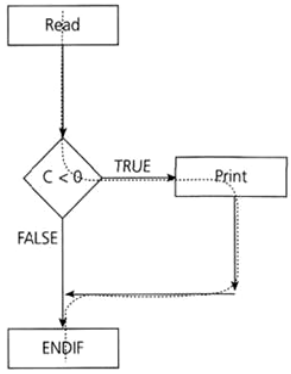


Branch/Decision/Edge Coverage:

- Each condition should be tested all possible outcomes at least once with TRUE and FALSE value. It covers both the true and false conditions.
- A branch is an outcome of decision.
- It is a more effective type of testing than statement coverage.
- You are able to validate all branches in the code and check the efficiency of the code.

$$\text{Decision coverage} = \frac{\text{Number of decision outcomes exercised}}{\text{Total number of decision outcomes}} \times 100\%$$

- **Example:** Read A
  Read B
  C=A–2*B
  if C<0 then
  print "C negative"
  end if



Test Case 1(True decision): A=20, B=15
Test Case 2(False decision): A=10, B=2

We draw the path taken by Test case 2; it would be a straight line from the read statement down the false exit and through the ENDIF. We have achieved 100% decision coverage if all the edges in the flow charts are covered by your test case 1.
100% decision coverage guarantees that 100% statement coverage but 100% statement coverage doesn't guarantee 100% decision coverage.

Condition Coverage or Predicate Coverage

- Condition coverage testing is carried out to check conditions which are generally Boolean expressions and provide test result in TRUE or FALSE.
- Condition coverage may or may not cover the entire decision coverage.
- In this process test only those conditions that return true or false are tested.
- Example: **IF ("X && Y")**

Test case 1: X=TRUE, Y=FALSE, Test case 2: X=FALSE, Y=TRUE

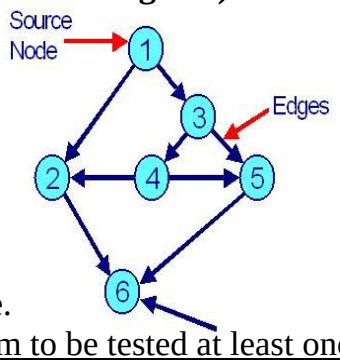
100% condition coverage doesn't guarantee that 100% decision coverage.

Path Testing (Path Coverage):

In path testing technique, we split a program into a number of different paths. A program or a part of a program can start from the beginning and take any of the paths to its completion. test cases are executed in that way, every path is executed at least once. In path testing each statement of program is guaranteed execute at least one time. Flow Graph, Cyclomatic Complexity and Graph Metrics are used to arrive at basis path.

Path Coverage = (Total Path executed) / (Total Number of Paths in Program) X 100

Some of the paths are

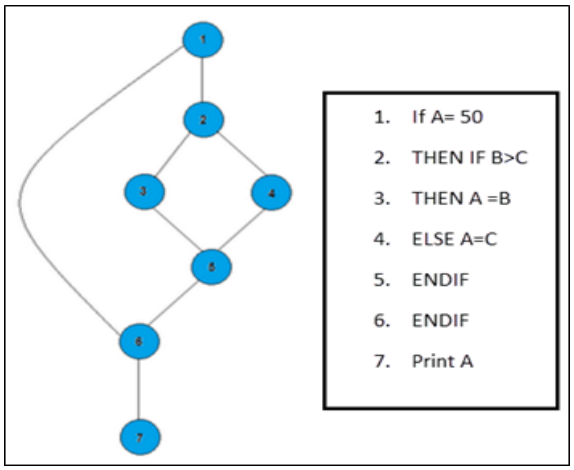


Path-1: 1 > 2 > 6
Path-2: 1 > 3 > 5 > 6
Path-3: 1 > 3 > 4 > 5 > 6
Path-4: 1 > 3 > 4 > 2 > 6

A software program have multiple entry and exit points.
Testing each point is challenging as well as time consuming.
To decrease unnecessary tests and achieve maximum test coverage.
The objective of path testing is all independent paths of the program to be tested at least once. i.e. maximizes the coverage of each test case.

Steps for Path testing

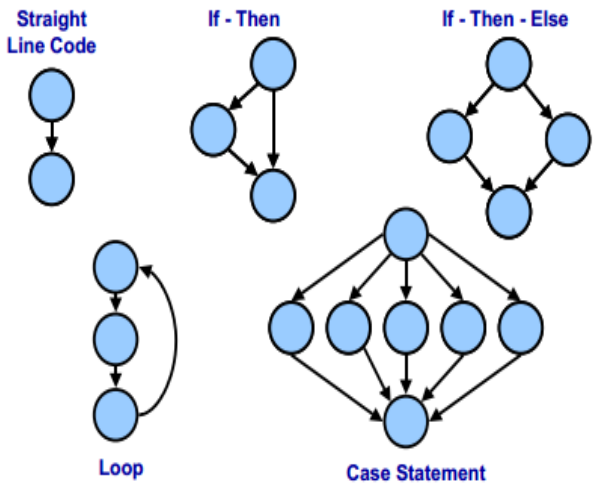
- Draw a control flow graph.
- Determine Cyclomatic complexity.
- Find a basis set of paths.
- Generate test cases for each path.

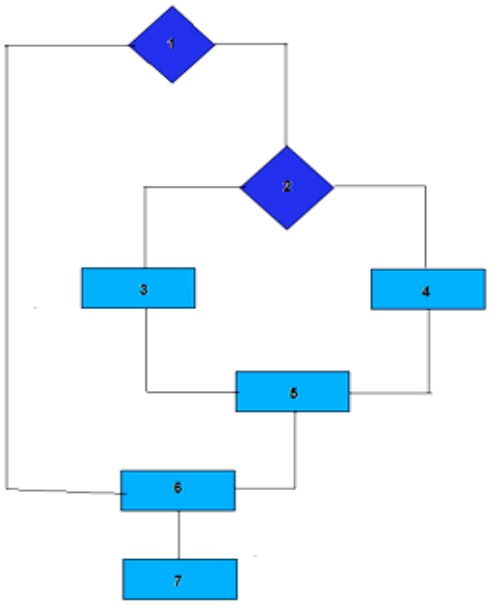


Step 1: Draw a control flow graph

Basic control flow graph structures:
- Arrows or edges represent flows of control.
-Circles or nodes represent actions.
- Areas bounded by edges and nodes are called regions.

- A predicate node is a node containing a condition.





Step 2: Determine Cyclomatic complexity

Cyclomatic complexity is software metric used to indicate the complexity of a program. It is a measure of the number of linearly independent paths.

There are several different methods to calculate Cyclomatic complexity:

1. **Cyclomatic complexity = edges - nodes + 2p**

   Where p = number of unconnected parts of the graph.

   From the example in **Step 1** we see that-

   The graph has 8 edges and 7 nodes

   The number of unconnected parts of the graph is 1

   so the **Cyclomatic complexity = 8-7+ 2*1= 3**

2. **Cyclomatic complexity= Number of Predicate Nodes + 1**

   from the example in **Step 1,**

   we can redraw it & show predicate nodes clearly:

   As we see, there are two predicate nodes in the graph.

   So the Cyclomatic complexity = 2+1= 3.

3. **Cyclomatic complexity =number of regions in the control flow graph**

   we have three regions in the control flow graph as below

   So the Cyclomatic complexity = 3.

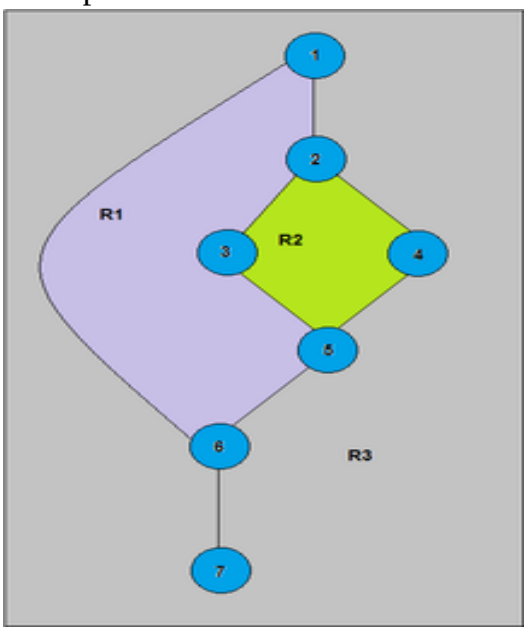
Step 3: Find a basis set of paths

The Cyclomatic complexity tells us the number of paths to evaluate for basis path testing. In the example, we have 3 paths, and our basis set of paths is:

Path 1: 1, 2, 3, 5, 6, 7.

Path 2: 1, 2, 4, 5, 6, 7.

Path 3: 1, 6, 7.



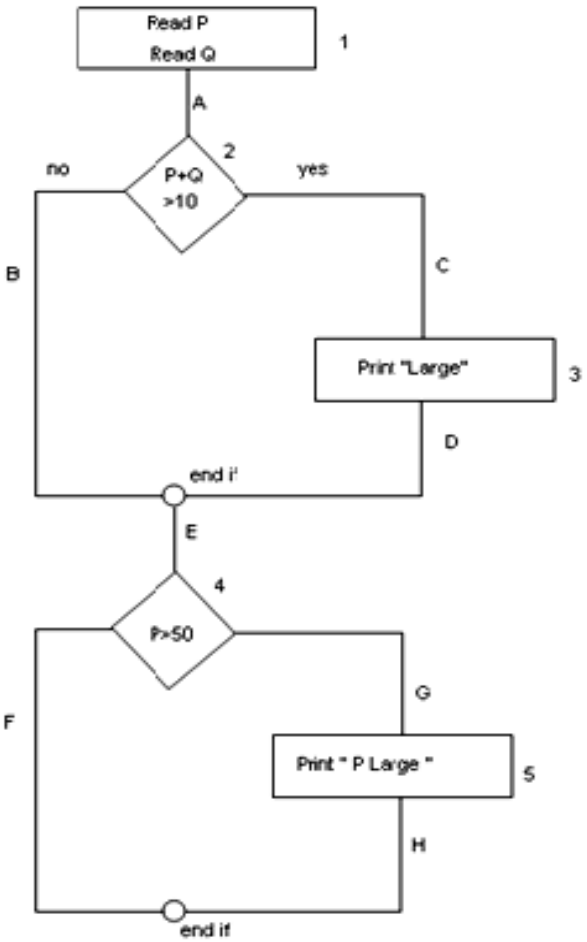
Step 4: Generate test cases for each path

after determining the basis set of path, we can generate the test case for each path. Usually we need at least one test case to cover one path. In the example, however, Path 3 is already covered by Path 1 and 2 so we only need to write 2 test cases.

- **Advantages of path testing:**
  - More focus on the program logic.
  - The flow of the program is analyzed
  - It helps to reduce the unnecessary tests
  - Test cases which exercise basis set will execute every statement in program at least once
- **Disadvantages of path testing:**
  - Expert testers are required.
  - It's not possible to test all the paths.

Example:

```
read p
read q
if p+q > 100 then
print "large"
endif
if p > 50 then
print "p large"
endif
```



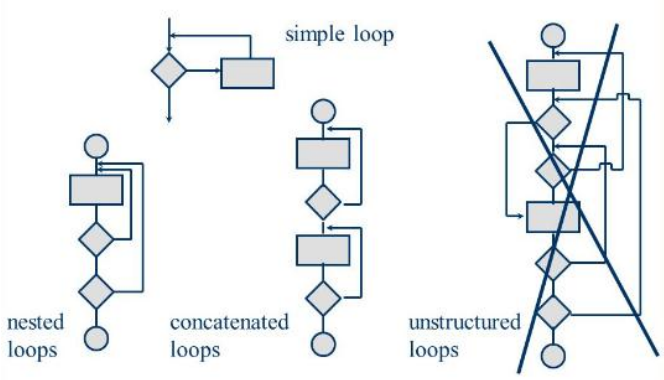
Statement Coverage: =1
1A-2C-3D-E-4G-5H
Branch Coverage: =2
1A-2C-3D-E-4G-5H,
1A-2B-E-4F
Path Coverage:=4
1A-2B-E-4F
1A-2B-E-4G-5H
1A-2C-3D-E-4G-5H
1A-2C-3D-E-4F

Loop Coverage Testing

Loop testing is used to check the accuracy of the loops in the program i.e. entirely concentrates on the validity of loop constructs. It is a part of Control Structure Testing for path testing, data validation testing, condition testing for wrong data, and condition testing for different situations.

Types of loop Testing

- Simple loop
- Nested loop
- Concatenated loop
- Unstructured loop



Why do Loop Testing?

- Helps to resolve the loop repetition issues
- Loops testing can expose performance/capacity blockage
- The loop's uninitialized variables can be detected by testing loops.
- It helps to identify loops initialization problems.

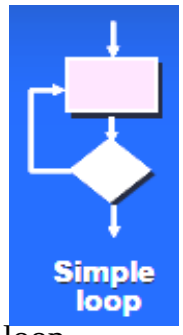
Testing Strategy of loop Testing:

While testing loop, it has to be checked at three different levels:

- When the loop is activated.
- When the loop is executed.
- When the loop is terminated.

➢ **Simple loop -Minimum conditions**

In simple loops normally added “for”, “while” or “do-while” loops in which we have given condition and loop run and terminates according to true and false condition result. This type of testing is performed basically to test the different conditions of the loop.



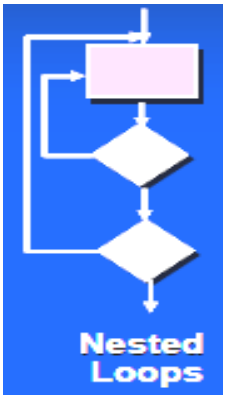
A simple loop is tested in the following way:

1. Skip the entire loop.
2. Make 1 pass through the loop.
3. Make 2 passes through the loop.
4. Make x passes through the loop,
   where x<y, n is the maximum number of passes through the loop.
5. Make "y","y-1","y+1" passes through the loop,
   where "y" is the maximum number of allowable passes through the loop.

➢ **Nested Loop**

To extend test approach, Loops within loops are known as nested loops. The numbers of tests required for testing nesting loops depends on the level of testing. This approach will help to reduce the number of tests:

1. Start at the innermost loop. Set all other loops to mini-mum values.
2. Conduct simple loop tests for the inner most loop while holding the outer loops at their loop counter values. Add other tests for out-of-range or excluded values.
3. Work outward, conducting tests for the next loop, but keeping all other outer loops at minimum values and other nested loops to “typical" values.
4. Continue until you have tested all loops.

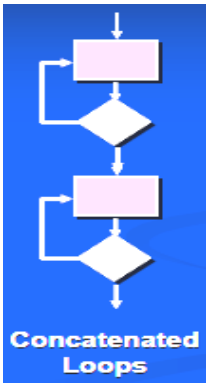


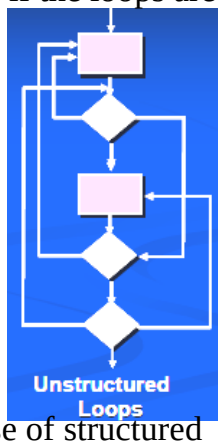
Example

```
If (n<5)
For (i=1; i<=n;i++)
{
Print (i)
}
```

➢ **Concatenated Loops**

The loops containing several loops that may be dependent or independent In case the loops are dependent on each other, the steps in nested loops are followed. On the other hand, if the loops are independent of each other, the steps in simple loops are followed.





Example:

```
int x[ ], y[ ];
for( int i; i <= 5, i++){
x[i] = i * 5;
}
 for( int j; j <= 5, j++){
y[j] = x[j] * j
}
```

➢ **Unstructured Loops**

Such loops of difficult to test; therefore, they should be redesigned so that the use of structured programming constructs can be reflected.

Static Testing

Static testing requires only the source code of software, not the binaries or executable files. Static testing checking defects in software without executing programs on computers. Also called as Dry-Run Testing. It is also used for examination of program's code and its associated documentation. Programmers read their own code manually for finding errors. Tester checks the code, design documents, requirement document and gives review comments on the work document. It starts early in the Life cycle and so it is done during the verification process.

This process has several advantages.

1. Humans can find errors that computers cannot.
   Example- when two variables with similar names used in program by mistake, then computer will not detect error and produce incorrect results where as a human being can spot such an error.
2. Multiple humans read and evaluate the program with multiple views and more problems openly identified than a computer.
3. A human evaluation of the code can compare with specifications or design but this is not possible, when a computer runs a test.
4. A human evaluation can detect many problems at one go and try to identify the root causes of the problems.
5. Humans test the code before execution, computer resources can be saved.
6. Defect is identified and corrected, Minimizes cost of defect fixing.
7. Finding defects later in the cycle, creates huge pressure on programmers fix defects with less time. Because of this pressure, there are higher chances of other defects sidling in.

Why Static Testing?

- To check code works according to the functional requirement
- To check any functionality has been missed out in code.
- The code handles errors properly.
- improve the quality of software by finding errors in early stages of the development cycle
- Reduce development time and increase development productivity
- Reduced testing cost and time

Static testing technique (Types of Review)

1. Code walkthrough
2. Code review
3. Code inspection

1. Code walkthrough

Walkthroughs and formal inspections are group-oriented methods (involve multiple people in the review process).Walkthrough is not a formal process. Walkthroughs are less formal than inspections. A set of people Checks program code and raise questions for the author. The author explains the logic of the code and answers the questions. If the author is unable to answer some questions, he then takes those questions and finds their answers.

- Author guides the review team via the document to fulfill the common understanding and collecting the feedback.
- A review team doesn’t require detailed study before meeting while authors are already in the scope of preparation.
- It is useful for higher-level documents i.e requirement specification and architectural documents.

2. Code review

Code review is systematic examination (known as peer review) of computer source code. It is planned to find and fix mistake in the initial development phase, improving both the overall quality of software and the developer’s skills. Reviews are vary from informal to formal review.

Informal Reviews

- Informal reviews are applied in the early stages of the life cycle of the document.
- These reviews are conducted between two person team. In later stages more people are involved.

- The aim of informal reviews is to improve the quality of the document and help the authors.
- These reviews are not based on the procedure and not documented.

Formal reviews

Formal reviews follow the formal process i.e reviews are well structured &managed.

3. Code Inspection (Formal inspection)

The focus of this method is to detect all faults, violations and other side-effects. A formal inspection should take place only when the author has made sure the code is ready for inspection by performing some basic desk checking and walkthroughs. When the code is in such a reasonable state of readiness, an inspection meeting is arranged.

The author or the moderator selects the review team. The chosen members have the skill sets to uncover as many defects as possible. In an introductory meeting, the inspectors get copies (hard copies or soft copies) of the code to be inspected along with other supporting documents such as the design document, requirements document, and any documentation of applicable standards.

There are four roles in inspection:

- **Author of the code-**The author also presents his view of what the program is planned to do along with any specific issues that he want inspection team to put extra focus on.
- **Moderator-** it expected to formally run the inspection according to the process. The moderator informs the team about the date, time & venue of the inspection meeting. The moderator takes sequentially program code, asking each inspector if there are any defects in that part of the code. If any of the inspectors raises a defect, then the inspection team reflects on the defect and, when agreed that there is a defect, classifies in two types — minor/major and systemic/mis-execution.
- **Inspectors-** The inspectors get adequate time to go through the documents and program and ascertain their compliance to the requirements, design, and standards. The inspection team assembles at the agreed time for the inspection meeting (also called the defect logging meeting). These are the people who actually provides, review comments for the code. There are typically multiple inspectors.
- **Scribe-** who takes detailed notes during the inspection meeting and circulates them to the inspection team after the meeting.

Defects found in the inspection meeting and the author fixing these defects. In case the defects are simple, the team may optionally call for a review meeting to inspect the fixes to ensure that they address the problems. In any case, defects found through inspection need to be tracked till completion and someone in the team has to verify that the problems have been fixed properly.

Technical Review

- Technical review is a discussion meeting that focuses on technical content of the document. **It is a less formal review.**
- It is guided by a trained moderator or a technical expert.
- The goal is to evaluate the value of technical concept in the project environment.
- Build the consistency in the use and representation of the technical concepts.
- In early stages it ensures that the technical concepts are used correctly.
- Notify the participants regarding the technical content of the document.

****

Questions:

1) What are the objective of software testing? Difference between manual and automation testing.
2) What is White box testing? Explain dynamic and static techniques.
3) Why Loop testing is needed? Explain in details.
4) Explain path testing with example.